

Spiking Neural P Systems with Weighted Synapses

Linqiang Pan · Xiangxiang Zeng · Xingyi Zhang ·
Yun Jiang

Published online: 14 October 2011
© Springer Science+Business Media, LLC. 2011

Abstract Spiking neural P systems are a class of distributed parallel computing models inspired from the way neurons communicate with each other by means of electrical impulses, where there is a synapse between each pair of connected neurons. However, in a biological system, there can be several synapses for each pair of connected neurons. In this study, inspired by this biological observation, synapses in a spiking neural P system are endowed with integer weight denoting the number of synapses for each pair of connected neurons. With the price of weight on synapses, quite small universal spiking neural P systems can be constructed. Specifically, a universal spiking neural P system with standard rules and weight having 38 neurons is produced as device of computing functions; as generator of sets of numbers, we find a universal system with standard rules and weight having 36 neurons.

Keywords Membrane computing · P system · Spiking neural small universal device

1 Introduction

Membrane computing is one of the recent branches of natural computing; it was initiated in [1] and has developed rapidly (already in 2003, ISI considered membrane computing as

L. Pan · X. Zeng (✉) · Y. Jiang
Department of Control Science and Engineering, Key Laboratory of Image Processing and Intelligent Control, Huazhong University of Science and Technology, Wuhan 430074, Hubei, China
e-mail: xzeng@foxmail.com

L. Pan
e-mail: lqpan@mail.hust.edu.cn

Y. Jiang
e-mail: jiangyun83@gmail.com

X. Zhang
Key Laboratory of Intelligent Computing and Signal Processing, School of Computer Science and Technology, Anhui University, Hefei 230039, Anhui, China
e-mail: xyzhanghust@gmail.com

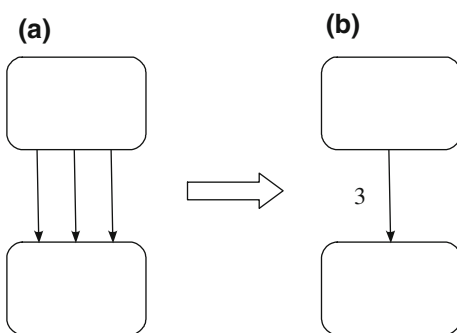
“fast emerging research area in computer science”, see <http://esi-topics.com>). The aim is to abstract computing ideas (data structures, operations with data, ways to control operations, computing models, etc.) from the structure and the functioning of a single cell and from complexes of cells, such as tissues and organs including the brain. The obtained models are distributed and parallel computing devices, usually called P systems. There are three main classes of P systems investigated: cell-like P systems [1], tissue-like P systems [2], neural-like P systems. Many variants of all these systems have been considered; an overview of the field can be found in [3] and [4], with up-to-date information available at the membrane computing website [5]. For a friendly introduction to membrane computing, one may consult [6] and [4]. The present article deals with a class of neural-like P systems, called *spiking neural P systems* (SN P systems, for short), introduced in [7].

SN P systems are a class of distributed and parallel computing devices, which are inspired by the neurophysiological behavior of neurons sending electrical impulses (spikes) along axons to other neurons. SN P systems can also be viewed as an evolution of P systems shifting from cell-like to neural-like architectures [5]. As we know, this biological background has already led to several models in the area of neural computation, e.g., see [8–10]. Roughly, an SN P system consists of a set of *neurons* placed in the nodes of a directed graph, where neurons send signals (*spikes*, denoted by the symbol a in what follows) along *synapses* (arcs of the graph). Thus, the architecture of an SN P system is that of a tissue-like P system, with only one kind of objects present in the cells. The objects evolve by means of *spiking rules*, which are of the form $E/a^c \rightarrow a; d$, where E is a regular expression over $\{a\}$ and c, d are natural numbers, $c \geq 1, d \geq 0$. In other words, a neuron containing k spikes such that $a^k \in L(E)$, $k \geq c$, can consume c spikes and produce one spike, after a delay of d steps. This spike is sent to all neurons connected by an outgoing synapse from the neuron where the rule was applied. There are also *forgetting rules*, of the form $a^s \rightarrow \lambda$, with the meaning that $s \geq 1$ spikes are forgotten if the neuron contains exactly s spikes. The system works in a synchronized manner: in each time unit, (1) the rule to be applied in each neuron is non-deterministically chosen, (2) a chosen rule must be applied for each neuron with applicable rules, (3) the work of the system is sequential in each neuron: only (at most) one rule is applied in each neuron. One of the neurons is considered to be the *output* neuron, and its spikes are also sent to the environment. The moments of time when a spike is emitted by the output neuron are marked with one, the other moments are marked with zero. This binary sequence is called the spike train of the system—it might be infinite if the computation does not stop. Various numbers can be associated with a *spike train*, which can be considered as *computed* (we also say *generated*) by an SN P system. In this study, the *result of a computation* is encoded by the time span elapsed between the first two consecutive spikes sent into the environment by the (output neuron of the) system.

In the usual variants of SN P systems, there is at most one synapse between two neurons, and the spikes are transported “loyally” (without any change) from one neuron to another neuron along the synapse. Recently, synapses are endowed with real number *weight* as a measure of the efficacy of synapse, and learning problem is considered in [11]. With different computer science and biological motivation, in this paper we focus on the number of synapses between neurons, and each connected pair of neurons are endowed with integer weight to denote the number of synapses between them.

The biological motivation of endowing synapse with integer weight comes from the following observation in neurobiology [12]. The number of synapses between two given neurons in a brain depends on multiple factors, including partitioning of 3D space by the dendrites of multiple neurons that share it, trajectories of the axons that navigate this 3D neuropil forest, and spatial distribution of chemical factors during axonal growth and neuronal activity.

Fig. 1 Pair of neurons connected by three synapses



Statistically, there are 1–6 synapses per connected pair for randomly chosen proximal pairs of excitatory neurons; inhibitory basket cells supposedly make 4–5 synapses onto each pyramidal cell which they contact. Figure 1a gives a pair of neurons with three synapses. For simplicity, the integer weight three is used to represent the number of synapses between this pair of neurons, as shown in Fig. 1b. In what follows, we use the form in Fig. 1b instead of that in Fig. 1a to represent the number of synapses between neurons. That is, a synapse is endowed with an integer weight to denote the number of synapses between the pair of neurons. When the weight is one, it is omitted in Figs. 3–11.

Looking for small universal computing devices of various types is a natural and well investigated topic in computer science, see, e.g., [13, 14], and the references therein. This topic was also considered in SN P systems [15], where a universal SN P system was obtained by using 84 neurons for standard rules as devices computing functions; for SN P systems used as generators of sets of numbers, a universal system with standard rules having 76 neurons was found. In this article, we investigate small universal SN P system with weighted synapses. As expected, with the price of weight on synapses, smaller universal SN P system can be devised in the sense that less neurons are used. Specifically, a universal spiking neural P system with standard rules and weight having 38 neurons is produced as device of computing functions; as generator of sets of numbers, we find a universal system with standard rules and weight having 36 neurons.

2 Prerequisites

Readers are assumed to have some familiarity with (basic elements of) language theory, e.g., from [16], as well as basic membrane computing [3] (for more updated information about membrane computing, please refer to [5]). We first introduce some notations and basic definitions, and then universal register machines.

For an alphabet V , V^* denotes the set of all finite strings over V , with the empty string denoted by λ . The set of all nonempty strings over V is denoted by V^+ . When $V = \{a\}$ is a singleton, then we write simply a^* and a^+ instead of $\{a\}^*$, $\{a\}^+$. The family of recursively enumerable languages is denoted by RE .

A regular expression over an alphabet V is defined as follows: (i) λ and each $a \in V$ is a regular expression, (ii) if E_1, E_2 are regular expressions over V , then $(E_1)(E_2)$, $(E_1) \cup (E_2)$, and $(E_1)^+$ are regular expressions over V , and (iii) nothing else is a regular expression over V . With each expression E we associate a language $L(E)$, defined in the following way: (i) $L(\lambda) = \{\lambda\}$ and $L(a) = \{a\}$, for all $a \in V$, (ii) $L[(E_1) \cup (E_2)] = L(E_1) \cup L(E_2)$,

$L[(E_1)(E_2)] = L(E_1)L(E_2)$, and $L[(E_1)^+] = L(E_1^+)$, for all regular expressions E_1, E_2 over V . Non-necessary parentheses are omitted when writing a regular expression, and also $(E)^+ \cup \{\lambda\}$ can be written as E^* .

A register machine is a construct $M = (m, H, l_0, l_h, I)$, where m is the number of registers, H is the set of instruction labels, l_0 is the start label, l_h is the halt label (assigned to instruction HALT), and I is the set of instructions; each label from H labels only one instruction from I , thus precisely identifying it. The instructions are of the following forms:

- $l_i : [\text{ADD}(r), l_j, l_k]$ (add one to register r and then go to one of the instructions with labels l_j, l_k non-deterministically chosen)
- $l_i : [\text{SUB}(r), l_j, l_k]$ (if register r is non-empty, then subtract one from it and go to the instruction with label l_j , otherwise go to the instruction with label l_k)
- $l_h : \text{HALT}$ (the halt instruction).

A register machine M computes (generates) a number n in the following way. The register machine starts with all registers empty (i.e., storing the number zero). It applies the instruction with label l_0 and proceeds to apply instructions as indicated by labels (and, in the case of SUB instructions, by the content of registers). If the register machine reaches the halt instruction, then the number n stored at that time in the first register r_0 is said to be computed by M . If the computation does not halt, then no number is generated. The set of all numbers computed by M is denoted by $N(M)$. It is known that register machines compute all sets of numbers which are Turing computable, hence they characterize NRE (see, e.g., [17]).

A register machine can also compute any Turing computable function: the arguments are introduced in specified register $r_1, \dots, r_k$ (without loss of the generality, we may assume that we use the first k registers), the machine starts with the instruction with label l_0 , and if it stops (with the instruction with label l_h), then the value of the function is placed in another specified register r_t , with all registers different from r_t being empty. The partial function computed in this way is denoted by $M(n_1, n_2, \dots, n_k)$. In the computing form, the register machines can be considered deterministic, without losing the Turing completeness; then, the ADD instructions $l_i : [\text{ADD}(r), l_j, l_k]$ have $l_j = l_k$ (and the instruction is written in the form $l_i : [\text{ADD}(r), l_j]$).

In [13], several universal register machines have been constructed for computing functions, with the input introduced in registers 1 and 2, and the result obtained in register 0. Here, we only introduce the specific universal register machine M_u used in [15], which is shown in Fig. 2.

3 SN P Systems with Weighted Synapses

We start by introducing the basic class of SN P systems—SN P systems without weight on synapses. For more details on these systems, please refer to [7].

A *spiking neural P system* of degree $m \geq 1$ is a construct of the form

$$\Pi = (O, \sigma_1, \dots, \sigma_m, \text{syn}, \text{in}, \text{out}), \text{ where:}$$

- $O = \{a\}$ is the singleton alphabet (a is called spike);
- $\sigma_1, \dots, \sigma_m$ are neurons, of the form

$$\sigma_i = (n_i, R_i), 1 \leq i \leq m, \text{ where:}$$

- a) $n_i \geq 0$ is the initial number of spikes contained in σ_i ;
- b) R_i is a finite set of rules of the following two forms:

Fig. 2 The universal register machine from [13]

$l_0 : (\text{SUB}(1), l_1, l_2),$	$l_1 : (\text{ADD}(7), l_0),$
$l_2 : (\text{ADD}(6), l_3),$	$l_3 : (\text{SUB}(5), l_2, l_4),$
$l_4 : (\text{SUB}(6), l_5, l_3),$	$l_5 : (\text{ADD}(5), l_6),$
$l_6 : (\text{SUB}(7), l_7, l_8),$	$l_7 : (\text{ADD}(1), l_4),$
$l_8 : (\text{SUB}(6), l_9, l_0),$	$l_9 : (\text{ADD}(6), l_{10}),$
$l_{10} : (\text{SUB}(4), l_0, l_{11}),$	$l_{11} : (\text{SUB}(5), l_{12}, l_{13}),$
$l_{12} : (\text{SUB}(5), l_{14}, l_{15}),$	$l_{13} : (\text{SUB}(2), l_{18}, l_{19}),$
$l_{14} : (\text{SUB}(5), l_{16}, l_{17}),$	$l_{15} : (\text{SUB}(3), l_{18}, l_{20}),$
$l_{16} : (\text{ADD}(4), l_{11}),$	$l_{17} : (\text{ADD}(2), l_{21}),$
$l_{18} : (\text{SUB}(4), l_0, l_h),$	$l_{19} : (\text{SUB}(0), l_0, l_{18}),$
$l_{20} : (\text{ADD}(0), l_0),$	$l_{21} : (\text{ADD}(3), l_{18}),$
$l_h : \text{HALT}$	

- (1) $E/a^c \rightarrow a^p; d$, where E is a regular expression over a , and $c \geq 1, d \geq 0, p \geq 1$, with the restriction $c \geq p$;
 - (2) $a^s \rightarrow \lambda$, for $s \geq 1$, with the restriction that for each rule $E/a^c \rightarrow a^p; d$ of type (1) from R_i , we have $a^s \notin L(E)$;
- $\text{syn} \subseteq \{1, 2, \dots, m\} \times \{1, 2, \dots, m\}$ with $i \neq j$ for each $(i, j) \in \text{syn}, 1 \leq i, j \leq m$ (synapses between neurons);
 - $\text{in}, \text{out} \in \{1, 2, \dots, m\}$ indicates the input and the output neurons, respectively.

If we always have $p = 1$ for all rules of the form $E/a^c \rightarrow a^p; d$, then the rules are said to be of the *standard* type; otherwise, they are called by *extended* rules.

The rules of type (1) are *firing* (we also say *spiking*) rules, and they are applied as follows. If the neuron σ_i contains k spikes, and $a^k \in L(E)$, $k \geq c$, then the rule $E/a^c \rightarrow a^p; d \in R_i$ can be applied. This means consuming (removing) c spikes (thus only $k - c$ spikes remain in neuron σ_i), the neuron is fired, and it produces p spikes after d time units (as usual in membrane computing, a global clock is assumed, marking the time for the whole system, hence the functioning of the system is synchronized). If $d = 0$, then these spikes are emitted immediately, if $d = 1$, then these spikes are emitted in the next step, etc. If the rule is used in step t and $d \geq 1$, then in steps $t, t + 1, \dots, t + d - 1$ the neuron is closed (this corresponds to the refractory period from neurobiology), so that it cannot receive new spikes (if a neuron has a synapse to a closed neuron and tries to send several spikes along it, then these particular spikes are lost). In the step $t + d$, the neuron spikes and becomes again open, so that it can receive spikes (which can be used starting with the step $t + d + 1$, when the neuron can again apply rules).

The rules of type (2) are *forgetting* rules; they are applied as follows: if the neuron σ_i contains exactly s spikes, then the rule $a^s \rightarrow \lambda$ from R_i can be used, meaning that all s spikes are removed from σ_i .

If a rule $E/a^c \rightarrow a; d$ has $E = a^c$, then we will write it in the simplified form $a^c \rightarrow a; d$.

If a rule $E/a^c \rightarrow a; d$ has $d = 0$, then we will write it in the simplified form $E/a^c \rightarrow a$.

In each time unit, if a neuron σ_i can use one of its rules, then a rule from R_i must be used. Since two firing rules, $E_1/a^{c_1} \rightarrow a^{p_1}; d_1$ and $E_2/a^{c_2} \rightarrow a^{p_2}; d_2$, can have $L(E_1) \cap L(E_2) \neq \emptyset$, it is possible that two or more rules can be applied in a neuron, and in that case, only one of them is chosen non-deterministically. Note however that, by definition, if a firing rule is applicable, then no forgetting rule is applicable, and vice versa.

Thus, the rules are used in the sequential manner in each neuron, at most one in each step, but neurons function in parallel with each other. It is important to notice that the applicability of a rule is established based on the total number of spikes contained in the neuron.

The initial configuration of the system is described by the numbers $n_1, n_2, \dots, n_m$, of spikes present in each neuron, with all neurons being open. During the computation, a configuration of the system is described by both the number of spikes present in each neuron and by the state of the neuron, more precisely, by the number of steps to count down until it becomes open (this number is zero if the neuron is already open). Thus, $\langle r_1/t_1, \dots, r_m/t_m \rangle$ is the configuration where neuron σ_i contains $r_i \geq 0$ spikes and it will be open after $t_i \geq 0$ steps, $i = 1, 2, \dots, m$; with this notation, the initial configuration is $C_0 = \langle n_1/0, \dots, n_m/0 \rangle$.

Using the rules as described above, one can define transitions among configurations. Any sequence of transitions starting in the initial configuration is called a computation. A computation halts if it reaches a configuration where all neurons are open and no rule can be used. In order to compute a function $f : N^k \rightarrow N$, we introduce k natural numbers $n_1, \dots, n_k$ in the system by “reading” from the environment a binary sequence $z = 10^{n_1-1} 10^{n_2-1} 1 \dots 10^{n_k-1} 1$. This means that the input neuron of Π receives a spike in each step corresponding to a digit 1 from the string z and no spike otherwise. Note that we input exactly $k + 1$ spikes, i.e., after the last spike we assume that no further spike is coming to the input neuron. The result of the computation is also encoded in the distance between two spikes: we impose the restriction that the system outputs exactly two spikes and halts (immediately after the second spike), hence it produces a spike train of the form $0^b 10^{r-1} 1$, for some $b \geq 0$ and with $r = f(n_1, \dots, n_k)$ (the system outputs no spike for a non-specified number of steps from the beginning of the computation until the first spike).

SN P systems with weighted synapses have the same components and the similar way of working as the above basic SN P systems except for the synapses. The weighted synapse is of the form

$$\text{syn} \subseteq \{1, 2, \dots, m\} \times \{1, 2, \dots, m\} \times \mathbb{N}.$$

For a synapse (i, j, r) , if p spikes are emitted by neuron σ_i , then $p \times r$ spikes will be received by neuron σ_j . Clearly, basic SN P system is a special case of SN P system with weighted synapses, where the weight of each synapse is one.

In the next sections, we will present an SN P system graphically, which is much easier to understand than the symbolic one. The neurons are placed in the nodes of a graph with the rules and the initial spikes in each neuron; the edges of the graph represent the synapses with weight denoting the number of synapses between each pair of connected neurons (when weight is one, it is omitted in the graph); the input neuron has an incoming arrow and the output neuron has an outgoing arrow, suggesting their communication with the environment.

4 A Small Universal SN P System in Computing Mode

Because basic SN P systems (without weight) have already been proved to be computationally universal in [7], this result is directly valid also for SN P systems with weighted synapses. Here, we look for small universal SN P Systems with weighted synapses.

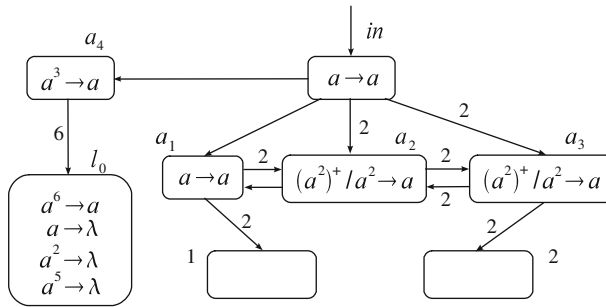
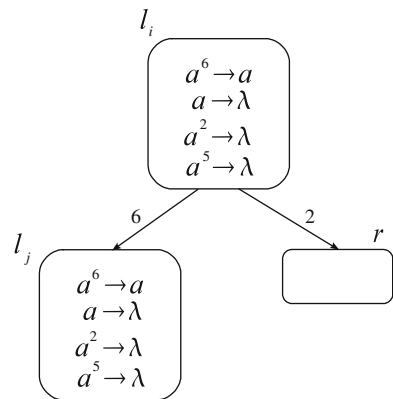

Fig. 3 INPUT Module

Fig. 4 ADD Module (simulating $l_i : [\text{ADD}(r), l_j]$)


Before constructing universal SN P systems, we modify the universal register machine M_u shown in Fig. 2 such that the register where we place the result is not subject to subtraction operations in the new register machine. We add a further register 8 to output the result, and replace the halt instruction of M_u with the following instruction:

$$l_h : (\text{SUB}(0), l_{22}, l'_h), \quad l_{22} : (\text{ADD}(8), l_h), \quad l'_h : \text{HALT}.$$

In this way, we obtain a new register machine M'_u which has 9 registers, 24 ADD and SUB instructions, and 25 labels.

In the following, a specific SN P system Π is constructed to simulate M'_u , which consists of four types of modules—INPUT module, ADD module, SUB module, and OUTPUT module.

- (i) An INPUT module is shown in Fig. 3, it is used for loading neurons σ_1 and σ_2 with $2g(x)$ and $2y$ spikes which represent the number $g(x)$ and y , respectively, and then activating neuron σ_{l_0} associated with the starting instruction of the register machine M'_u by introducing 12 spikes in it.
- (ii) An ADD module for simulating a deterministic ADD instruction: $l_i : (\text{ADD}(r), l_j)$ is presented in Fig. 4.
- (iii) A SUB module for simulating a SUB instruction $l_i : [\text{SUB}(r), l_j, l_k]$ is shown in Fig. 5. Besides four neurons for three labels and one register, each such module contains one auxiliary neuron σ_{b_i} .
- (iv) An OUTPUT module for outputting the result placed in register 8 is shown in Fig. 6.

Fig. 5 SUB Module (simulating $l_i : [\text{SUB}(r), l_j, l_k]$)

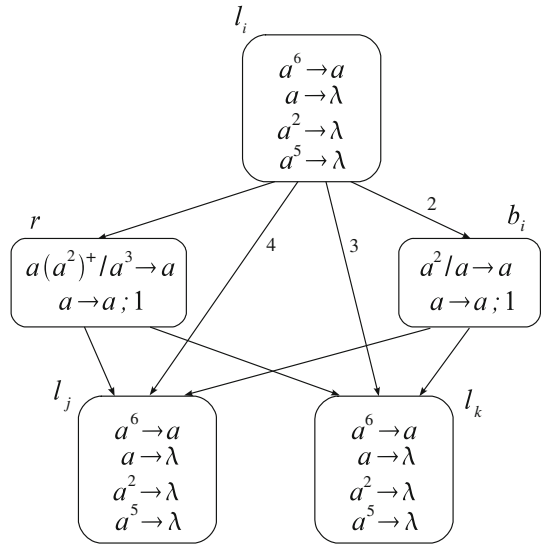
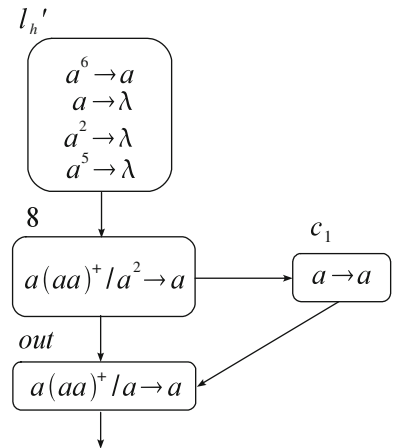


Fig. 6 OUTPUT Module



The simulation of register machine M'_u by SN P system Π is done as follows. Neurons are associated with each register and with each label of an instruction of the machine, where INPUT module contains contains four auxiliary neurons $\sigma_{a_1}, \sigma_{a_2}, \sigma_{a_3}, \sigma_{a_4}$ which do not correspond to registers or labels of instructions; each SUB module contains an auxiliary neuron σ_{b_1} ; OUTPUT module contains an auxiliary neuron σ_{c_1} . If a register contains a number n , then associated neuron will contain $2n$ spikes.

The input of the system is encoded to the spike train $10^{g(x)-1}10^{(y-1)}1$. In the initial configuration of Π all neurons are empty. The neuron σ_{in} of the INPUT module reads the spike train $10^{g(x)-1}10^{(y-1)}1$, neuron σ_1 receives from neuron σ_{a_1} as many spikes as the double number of steps between the first two input spikes (the number of spikes sent by neuron σ_{a_1} is amplified by the synapse weight two), and after that neuron σ_{a_1} gets “over flooded” by the second input spike and is blocked. Neuron σ_{a_3} starts to send spikes to neuron σ_2 after collecting two spikes and stop sending after receiving the third spike (the number of spikes sent out by neuron σ_{a_3} is also amplified by the synapse weight two). In this way, $2g(x)$ and

2y spikes are loaded in neurons σ_1 and σ_2 , respectively. When six spikes are introduced in neuron σ_{l_0} (associated with the starting instruction of the register machine), the simulation work of the system is triggered. In general, the simulation of an ADD or SUB instruction starts by introducing six spikes in the neuron with the instruction label. As we will see, when an instruction is simulated by using ADD or SUB module, the system cannot activate the simulation of another instruction at the same time.

The instruction with label l_0 is a SUB instruction. Assume that we are in a step t when we have to simulate a SUB instruction $l_i : [\text{SUB}(r), l_j, l_k]$ by the SUB module in Fig. 5, with six spikes present in neuron σ_{l_i} and no spike in any other neuron, except those neurons associated with the registers. At step $t + 1$, neuron σ_{l_i} fires, neuron σ_r receives one spike, neuron σ_{l_j} receives 4 spikes, neuron σ_{l_k} receives 3 spikes, and neuron σ_{b_i} receives 2 spikes. At step $t + 2$, neuron σ_{b_i} fires and send one spike to neuron σ_{l_j} and σ_{l_k} , respectively. For neuron σ_r , there are two cases.

- (i) If the number of spikes in neuron σ_r at step t is 0, then the rule $a \rightarrow a; 1$ consumes the single spike present in the neuron and fires at step $t + 3$.
- (ii) If the number of spikes in neuron σ_r at step t is $2k$ with $k > 0$, then the rule $a(a^2)^+/a^3 \rightarrow a$ consumes three spikes and fires at step $t + 2$.

Note that both rules present in neuron σ_r consume an odd number of spikes and once a rule is applied, no other rule in this neuron is enabled anymore until the simulation of next instruction dealing with register r .

In the case (i), at step $t + 1$ neuron σ_{l_k} receives three spikes; at step $t + 2$ one spike from neuron σ_b reach neuron σ_{l_k} , now neuron σ_{l_k} contains four spikes and has no rule to apply; at step $t + 3$, one spike from neuron σ_r and one spike from neuron σ_b reach neuron σ_{l_k} ; at step $t + 4$, having six spikes inside, neuron σ_{l_k} fires.

In the case (ii), at step $t + 1$ neuron σ_{l_j} contains four spikes; at step $t + 2$, one spike from neuron σ_r and one spike from neuron σ_b reach neuron σ_{l_j} ; at step $t + 3$, having six spikes inside, neuron σ_{l_j} fires.

We start from σ_{l_i} and ended in σ_{l_j} if the register is non-empty and decreased by one, and in σ_{l_k} if the register is empty. So the simulation of the SUB instruction is correct.

Let us check now Fig. 4, starting from the situation of having six spikes in neuron σ_{l_i} and no spikes in other neurons, except neuron σ_r , which holds even number of spikes (half of this number is the value of the corresponding register r). Having six spikes inside, neuron σ_{l_i} spikes. Neuron σ_r receives two spikes, so the number of spikes in neuron σ_r increases by two. Neuron σ_{l_j} receives six spikes, and in the next step it fires. So the the ADD instruction $l_i : [\text{ADD}(r), l_j]$ is correctly simulated by ADD module.

Recall that the result of the computation is in register 8, which is never decremented during the computation. We output the result by means of OUTPUT module in Fig. 6. When neuron $\sigma_{l'_h}$ receives six spikes, it fires and neuron σ_8 receives one spike; in this way, the number of spikes in this neuron becomes odd, hence its rule can be applied. A spike arrives in the output neuron, which spikes. From the next step on until exhausting the spikes from neuron σ_8 , the output neuron receives two spikes (one from σ_8 and one from σ_{c_1}), hence neuron σ_{out} collects an even number of spikes and cannot fire. When neuron σ_8 has only one spike, it cannot fire anymore, but one further spike comes in neuron σ_{out} from neuron σ_{c_1} , and thus the output neuron spikes for the second time (which is the last time).

In general, the system contains
 9 neurons for the 9 registers,
 25 neurons for the 25 labels,

Fig. 7 A module for consecutive ADD instructions

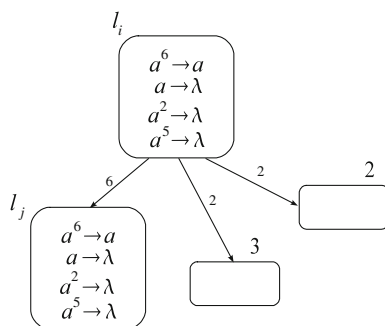
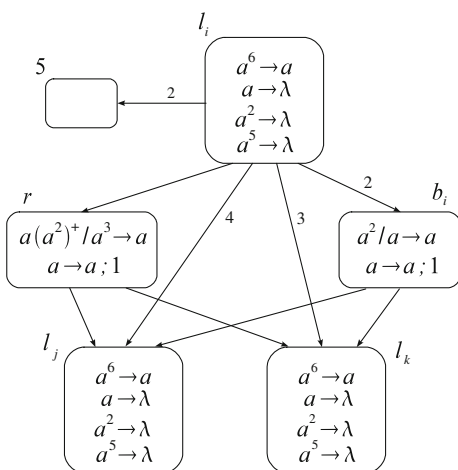


Fig. 8 A module for consecutive ADD–SUB instructions



- 5 neurons in the INPUT module,
- 14 neurons for all the 14 SUB instructions,
- 2 neurons in the OUTPUT module.

So there are 55 neurons in total.

The number of neurons can be decreased by exploiting some particularities of the register machine M'_u . We just give the corresponding modules for decreasing the number of neurons, and do not go into details to explain the work of the modules. The reader can check it as above (if necessary, the reader can also consult [7] and [15] in this respect).

One method to decrease the number of neurons is using some “code optimization” as in [15]. Let us observe that the sequence of two consecutive ADD instructions

$$l_{17} : [\text{ADD}(2), l_{21}], \quad l_{21} : [\text{ADD}(3), l_{18}],$$

without any other instruction addressing the label l_{21} , can be simulated by the module from Fig. 7, and in this way we save the neuron associated with l_{21} .

A similar module as that from Fig. 8 is possible for each of the following two sequences of ADD–SUB instructions, where we can save two neurons for the labels l_6 and l_{10} :

$$\begin{aligned} l_5 & : [\text{ADD}(5), l_6], & l_6 & : [\text{SUB}(7), l_7, l_8] \\ l_9 & : [\text{ADD}(6), l_{10}], & l_{10} & : [\text{SUB}(4), l_0, l_{11}]. \end{aligned}$$

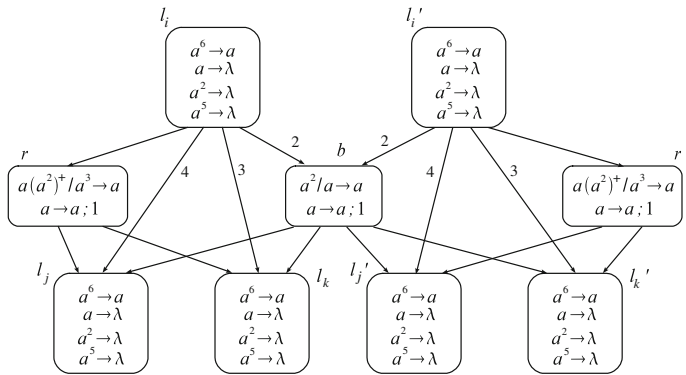


Fig. 9 A module for two SUB instructions

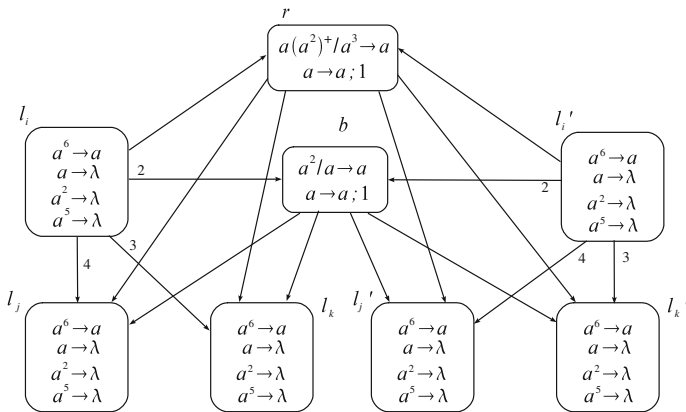


Fig. 10 A module for the SUB instructions associated with the same register

We can decrease the number of auxiliary neurons used in SUB modules by the observation that all SUB modules can share one auxiliary neuron. For any two SUB modules associated with instructions $l_i : [\text{SUB}(r), l_j, l_k]$ and $l_i' : [\text{SUB}(r'), l_j', l_k']$ respectively, we show how the two SUB modules share one auxiliary neuron σ_b in Fig. 9. Let us examine the work of the two SUB modules in detail: assume that we start to simulate the SUB instruction $l_i : [\text{SUB}(r), l_j, l_k]$ at time t , hence neuron σ_b will send spikes to neurons $\sigma_{l_j'}$ and $\sigma_{l_k'}$ at time $t + 1$ and $t + 2$. However, the number of spikes accumulate in either neurons $\sigma_{l_j'}$ or $\sigma_{l_k'}$ is less than two both at time $t + 1$ and $t + 2$, so these spikes will be forgotten immediately because of the rules $a \rightarrow \lambda$ and $a^2 \rightarrow \lambda$. Hence, after the simulation of $l_i : [\text{SUB}(r), l_j, l_k]$, all the neurons except neuron σ_r will go back to their initial state. The next simulation can continue correctly.

In particular, for two SUB modules dealing with the same register, as shown in Fig. 10, we can also prove it works correctly in a similar way.

From the above description about the decreasing of the number of neurons, we have

9 neurons	for the 9 registers,
21 neurons	for the 21 labels, (three neurons are saved in ADD-ADD and ADD-SUB modules),
5 neuron	in the INPUT module,
1 neuron	for the 14 SUB instructions,
2 neurons	in the OUTPUT module.

So there are 38 neurons in total.

Theorem 1 *There is a universal computing SN P system with standard rules and weighted synapses having 38 neurons.*

5 A Small Universal SN P System as Number Generator

In this section, we consider small universal SN P systems with weighted synapse as number generators. In short, a generating SN P system Π_u is universal if, given a fixed admissible enumeration of the unary partial recursive functions, $(\varphi_0, \varphi_1, \dots)$, there is a recursive function g such that for each natural number x , if we input the number $g(x)$ in Π_u , the set of numbers generated by the system is equal to $\{n \in N \mid \varphi_x(n) \text{ is defined}\}$. Otherwise stated, after introducing the “code” $g(x)$ of the partial recursive function φ_x in a specified neuron, the system generates all number n for which $\varphi_x(n)$ is defined.

Different from the universal computing devices considered in Sect. 4, the strategy to simulate universal number generators is the following: first, the system reads the string $10^{g(x)-1}1$ from the environment and loads $2g(x)$ spikes in neuron σ_1 . Second, the system loads neuron σ_2 non-deterministically with an arbitrary natural number n (by introducing $2n$ spikes); at the same time, outputs the number n (by having neuron σ_{out} output the spike train $10^{n-1}1$). Third, neuron l_0 is activated to start simulating the register machine M_u from Fig. 2, with $g(x)$ in register 1 and n in register 2. If the computation in M_u halts, then also the computation in system Π_u halts. It means that the function φ_x is defined for n , hence n is introduced in the set of generated numbers.

Because of the difference between number generating and function computing, we have to make some modification of the system constructed in Sect. 4.

- (1) We simulate the register machine M_u (not M'_u). The additional register 8 can be omitted, because output module is not necessary.
- (2) The label l_h can be omitted by simplify the instruction $l_{18} : [\text{SUB}(4), l_0, l_h]$, just ignoring l_h and halting.
- (3) The input module should be combined with the output one, at the same time non-deterministically producing the number n . The combined INPUT-OUTPUT module is presented in Fig. 11.

We do not go into the details of the work of the modified system, and directly give the number of neurons. The modified system contains

8 neurons	for the 8 registers,
19 neurons	for the 19 labels (l_h is saved, and three labels are saved in ADD-ADD and ADD-SUB modules),
1 neuron	for the 13 SUB instructions,
8 neurons	in the INPUT-OUTPUT module.

So there are 36 neurons in total.

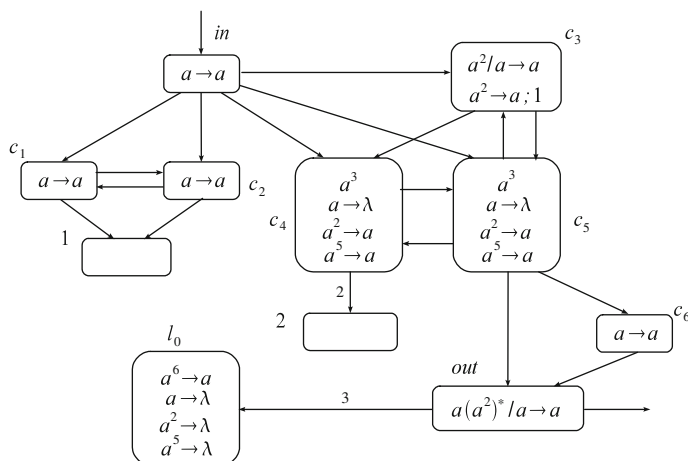


Fig. 11 An INPUT-OUTPUT module

Table 1 A computation of the INPUT-OUTPUT module from Fig. 11, where $g(x) = 2$, $n = 3$

Step	σ_{in}	c_1	σ_{c_2}	σ_{c_3}	σ_{c_4}	σ_{c_5}	σ_{c_6}	σ_1	σ_2	σ_{out}	σ_{l_0}
$t = 1$	1	0	0	0	3	3	0	0	0	0	0
$t = 2$	0	1	1	1	4	4	0	0	0	0	0
$t = 3$	1	1	1	1	4	4	0	2	0	0	0
$t = 4$	0	2	2	$2(a^2/a \rightarrow a)$	5	5	0	4	0	0	0
$t = 5$	0	2	2	$2(a^2/a \rightarrow a)$	2	2	1	4	2	1(fire)	0
$t = 6$	0	2	2	$2(a^2/a \rightarrow a; 1)$	2	2	1	4	4	2	3
$t = 7$	0	2	2	0(close)	1	1	1	4	6	4	3
$t = 8$	0	2	2	0	1	1	0	4	6	5(fire)	3
$t = 9$	0	2	2	0	0	0	0	4	6	4	6

Bold numbers indicate the input spike train 101

Theorem 2 By using weighted synapses, there is a universal number generating spiking neural P system with standard rules having 36 neurons.

Although a description about the work of SN P systems is already given in Sect. 4, for more clearly understanding how SN P systems work, we take the INPUT-OUTPUT module presented in Fig. 11 as an example and give a step by step analysis of a possible computation. Here, we assume $g(x) = 2$ and introduce the string 101 into neuron σ_{in} . The computation of the module is shown in Table 1, where it takes nine steps, with spikes sent out by the output neuron at steps 5 and 8 (hence a natural number $n = 3$ is generated). As a result, four spikes and six spikes are loaded in neuron σ_1 and σ_2 , respectively, neuron σ_{l_0} is activated at step 9.

6 Conclusions and Remarks

There can be several synapses for each pair of connected neurons in brain. Inspired by this biological observation, synapses in SN P systems are endowed with integer weight denot-

ing the number of synapses for each pair of connected neurons. With the price of weight on synapses, we can construct quite small universal SN P systems. Specifically, a universal spiking neural P system with standard rules and weight having 38 neurons is produced as device of computing functions; as generator of sets of numbers, we find a universal system with standard rules and weight having 36 neurons. The result is interesting in the sense that in the theoretical framework of SN P systems there exists quietly small universal “brains”.

With different biological and computer science motivation, in [18], computable real numbers were used as weight on synapses. It is a hot topic to digitize or quantize synaptic weight to be integers from real numbers in the field of neural network. However, in the framework of SN P systems, little work was done about systems computing real numbers. It is of interest to investigate what results can be obtained with the feature of real numbers weight on synapses. In particular, it deserves to be investigated whether the feature of real numbers weight on synapses is useful for applications of SN P systems in learning and pattern recognition.

In basic SN P systems (synapses have no weight), the number of spikes cannot be amplified by synapses. It has been proved in [19] that a deterministic basic SN P system of polynomial size cannot solve an NP-complete problem in a polynomial time unless $P = NP$. However, in SN P systems with weighted synapses, the number of spikes can be amplified by synapses. Does it enhance the efficiency of SN P system to solve computationally hard problem using the amplification of the number of spikes as a computing resource? It remains an open problem whether a deterministic SN P system with weighted synapses and polynomial size can solve an NP-complete problem in a polynomial time.

Acknowledgements This study was supported by National Natural Science Foundation of China (61033003 and 30870826), Ph.D. Programs Foundation of Ministry of Education of China (20100142110072), Fundamental Research Funds for the Central Universities (2010ZD001). A series of suggestions made by the anonymous referees, who carefully read the paper, are gratefully acknowledged.

References

1. Păun G (2000) Computing with membranes. *J Comput Syst Sci* 43(1):108–143
2. Martín-Vide C, Pazos J, Păun G, Rodríguez-Patón A (2003) Tissue P systems. *Theor Comput Sci* 296(2):295–326
3. Păun G (2002) Membrane computing. An introduction. Springer-Verlag, Berlin
4. Păun G, Rozenberg G, Salomaa A (eds) (2010) Handbook of membrane computing. Oxford University Press, Oxford
5. The P System Web Page (2011). <http://ppage.psysteams.eu>. Accessed 2 July 2011
6. Păun G, Rozenberg G (2002) A guide to membrane computing. *Theor Comput Sci* 287(1):73–100
7. Ionescu M, Păun G, Yokomori T (2006) Spiking neural P systems. *Fundamenta Informaticae* 71(2–3): 279–308
8. Gerstner W, Kistler W (2002) Spiking neuron models. Single neurons, populations, plasticity. Cambridge University Press, Cambridge
9. Maass W (2002) Computing with spikes. *Spec Issue Found Inform Process TELEMATIK* 8(1):32–36
10. Maass W, Bishop C (eds) (1999) Pulsed neural networks. MIT Press, Cambridge
11. Gutiérrez-Naranjo MA, Pérez-Jiménez MJ (2009) Hebbian learning from spiking neural P systems view. *LNCS* 5391:217–230
12. Bear MF, Connors BW, Paradiso MA (2007) Neuroscience: exploring the brain. Lippincott Williams & Wilkins, Philadelphia
13. Korec I (1996) Small universal register machines. *Theor Comput Sci* 168:267–301
14. Rogozhin Y (1996) Small universal turing machines. *Theor Comput Sci* 168:215–240
15. Păun A, Păun G (2007) Small universal spiking neural P systems. *BioSystems* 90(1):48–60
16. Rozenberg G, Salomaa A (eds) (1997) Handbook of formal languages, 3 volumes. Springer-Verlag, Berlin
17. Minsky M (1967) Computation—finite and infinite machines. Prentice Hall, New Jersey

18. Wang J, Hoogeboom HJ, Pan L, Păun G, Pérez-Jiménez MJ (2010) Spiking neural P systems with weights. *Neural Comput* 22(10):2615–2646
19. Leporati A, Zandron C, Ferretti C, Mauri G (2009) On the computational power of spiking neural P systems. *Int J Unconvent Comput* 5(5):459–473